

Cahier API — Développeur Mobile

Contrat API pour l'application mobile TG-Market (AK Market). Base URL : <https://tg-market-api-production.up.railway.app/api> Socket : <wss://tg-market-api-production.up.railway.app> (Socket.IO v4)

1. Authentification

- **JWT Bearer** : header `Authorization: Bearer <accessToken>` sur toutes les routes protégées (✓).
- Login : `POST /auth/login { email, password } → { accessToken, refreshToken, user }` (accès 24h, refresh 30j).
- Refresh : `POST /auth/refresh` avec `refreshToken` (body ou cookie).
- Socket : `io(url, { auth: { token: accessToken } })` — sans token → `connect_error: "Authentication error"`.
- **Remarque** : sans « se souvenir de moi », le token est stocké côté client ; le socket doit se reconnecter après chaque login et après un refresh de token.

2. Objet Escrow (structure retournée partout)

`GET /escrow`, `GET /escrow/:id` et toutes les mutations retournent cet objet :

```
{
  "id": 12, // number
  "productId": 34, // number | null
  "bundleId": null, // number | null (lot acheté)
  "buyerId": 8,
  "sellerId": 15,
  "buyerName": "Jean Kouassi",
  "sellerName": "Ama Mensah",
  "productTitle": "iPhone 12",
  "productImage": "https://res.cloudinary.com/.../img.jpg",
  "bundleTitle": null,
  "amount": 250000, // FCFA, montant produit
  "fee": 2500, // frais plateforme (prélevés sur le vendeur)
  "buyerFee": 1250, // frais d'achat payé par l'acheteur
  "status": "paid", // voir §3
  "paymentMethod": "floozy", // string | null
  "confirmationToken": "uuid", // payload du QR (voir §6)
  "payout": null, // objet payout | null (voir §7)
  "createdAt": "2026-08-14T10:00:00.000Z",
  "confirmedAt": null, // date paiement confirmé | null
  "releasedAt": null, // date fonds libérés | null
  "disputedBy": null // id de l'auteur du litige | null – NOUVEAU
}
```

3. Statuts escrow

Status	Signification
<code>pending</code>	En attente de paiement
<code>awaiting_verification</code>	Paieement manuel en cours de vérification
<code>paid</code>	Paieement confirmé, vendeur doit expédier
<code>pending_delivery</code>	Expédiée, acheteur doit confirmer la réception
<code>delivered</code>	Livrée — confirmation par code requise
<code>disputed</code>	Litige en cours
<code>completed</code>	Terminée, fonds libérés au vendeur
<code>cancelled</code>	Annulée
<code>refunded</code>	Remboursée

4. Cycle de vie de la commande (séquentiel)

#	Étape	Endpoint	Rôle	Résultat
1	Créer	<code>POST /escrow { productId, sellerId, paymentMethod }</code>	Acheteur	statut <code>pending</code>
2	Payer	<code>POST /escrow/:id/confirm-payment</code> (ou <code>POST /payments/initiate</code> Flutterwave)	Acheteur	<code>awaiting_verification</code> ou <code>paid</code>
3	Expédier	<code>PUT /escrow/:id/mark-shipped</code>	Vendeur	<code>pending_delivery</code>
4	Confirmer réception	<code>PUT /escrow/:id/confirm-delivery</code>	Acheteur	<code>delivered</code> + code 4 chiffres généré
5	Saisir le code	<code>PUT /escrow/:id/confirm-code { code }</code>	Vendeur	<code>completed</code> , payout au vendeur

L'acheteur peut aussi scanner le QR du vendeur : `POST /escrow/scan-confirm { token }` → `{ confirmationCode }` (code à 4 chiffres à communiquer au vendeur).

5. Litiges — endpoints (NOUVEAUX/COMPLÉTÉS)

5.1 Ouvrir un litige

`PUT /escrow/:id/dispute` — Auth ✓ — Acheteur **ou** vendeur

```
{ "reason": "Produit non conforme (min. 10 caractères)" }
```

- → 200 objet escrow avec `status: "disputed"`, `disputedBy: <id auteur>`
- Interdit si `completed` / `cancelled` / `refunded` → 400
- Non participant → 403

5.2 Reformer son litige (reprenre la vente) — NOUVEAU

PUT `/escrow/:id/cancel-dispute` — Auth ✓ — **Seul l'auteur du litige**

- Body : aucun
- → 200 escrow avec `status` restauré (`paid`, `pending_delivery` ou `delivered`), `disputedBy: null`
- Erreurs :
 - 404 « Transaction introuvable »
 - 403 « Seul l'auteur du litige peut le reformer »
 - 400 « Ce litige a déjà été traité » (statut ≠ `disputed`)
 - 400 « Ce litige ne peut pas être repris automatiquement, contactez l'administrateur » (litige ancien, pré-migration)

5.3 Résolution par l'admin

PUT `/admin/escrow/:id/resolve` — Auth admin ✓

```
{ "action": "refund" }      // remboursement acheteur → refunded
{ "action": "complete" }   // payout vendeur → completed
```

- 400 si action invalide ou transaction non `disputed`

5.4 Reprise par l'admin — NOUVEAU

PUT `/admin/escrow/:id/resume` — Auth admin ✓

- Body : aucun — même comportement que §5.2 (statut restauré)
- Notifie acheteur + vendeur

5.5 Annuler la commande

PUT `/escrow/:id/cancel` — Auth ✓ — 400 si déjà traitée

6. QR code & code de confirmation

- Le QR encode `confirmationToken` (UUID de l'escrow, exposé dans l'objet escrow).
- POST `/escrow/scan-confirm { token }` (Auth ✓, réservé à l'acheteur) → `{ confirmationCode }` (4 chiffres).
- PUT `/escrow/:id/confirm-code { code }` (Auth ✓, réservé au vendeur) :
 - 5 tentatives max (compteur Redis 1h) puis code invalidé → le vendeur doit recontacter l'acheteur.
 - 400 « Code invalide » sur erreur.
- Payout vendeur automatique à la confirmation (montant – frais). Badge `first_sale` accordé à la première vente complétée.

7. Payout (vue vendeur)

`escrow.payout` (quand présent) :

```
{
  "id": 5,
  "amount": 247500,           // net vendeur
  "status": "pending",       // pending | paid | failed | retrying
  "method": "flooze",        // flooz | tmoney | bank | flutterwave
  "errorMessage": null,
  "createdAt": "..."
}
```

8. Événements Socket.IO à écouter

Événement	Payload	Quand
<code>escrow_updated</code>	<code>{ escrow }</code>	Tout changement de statut (paiement, expédition, litige, reprise, résolution) → refetch <code>GET /escrow/:id</code>
<code>escrow_created</code>	<code>{ escrow }</code>	Nouvelle commande
<code>new_message</code>	<code>{ conversationId, message }</code>	Message reçu dans une conversation (room <code>conversation:<id></code>)
<code>message_notification</code>	<code>{ conversationId }</code>	Message reçu hors conversation ouverte (badge)
<code>notification</code>	<code>{ id, type, title, description, productId, read, createdAt, metadata }</code>	Notification temps réel
<code>user_online / user_offline</code>	<code>{ userId }</code>	Statut présence
<code>user_typing / user_stop_typing</code>	<code>{ conversationId, userId }</code>	Indicateur de saisie
<code>wallet_updated</code>	—	Solde changé (remboursement, payout) → refetch wallet

Événements client : `join_room/leave_room` (`{ room: "conversation:<id>" }`), `send_message` (`{ conversationId, content, type?, metadata? }`), `typing_start/typing_stop`, `notification_read`.

9. Types de notifications (champ `type`)

Type	Signification
payment_confirmed	Paiement confirmé (vendeur)
delivery_confirmed	Livraison confirmée
escrow_disputed	Litige ouvert sur une commande
escrow_dispute_cancelled	Litige refermé, vente reprise — NOUVEAU
dispute_resolved	Litige tranché par l'admin (refund ou complete)
escrow_cancelled	Commande annulée
message	Nouveau message

10. Erreurs — format standard

```
{ "error": "message lisible", "errors": { "body": ["détail validation"] } }
```

- 400 requête invalide / action impossible · 401 token manquant/invalide · 403 non autorisé · 404 introuvable · 422 validation Zod (avec `errors.body`).

11. Endpoints admin (rôle admin uniquement)

Méthode	Path	Description
GET	/admin/escrow	Liste paginée (inclut <code>paymentRef</code> pour vérifier les paiements manuels)
PUT	/admin/escrow/:id/verify	Vérifier paiement manuel
PUT	/admin/escrow/:id/resolve	{ action: "refund" "complete" }
PUT	/admin/escrow/:id/resume	Reprendre la vente — NOUVEAU
PUT	/admin/products/:id/status	Statut produit — 400 si <code>sold</code> → <code>active</code> (produit vendu)

12. Liens utiles

- Frontend web : <https://ak-market.pages.dev>
- Health check : `GET /health`
- Test socket : connexion avec token puis `join_room` sur `conversation:<id>`

13. Push Notifications — /api/push (ajouté août 2026)

Fonctionne pour les PWA installées (Web Push via VAPID). Le backend envoie le push à chaque notification (`notifyUser`) et à chaque message (API `sendMessage` + socket `send_message`, `tag: msg-{conversationId}` pour remplacer la précédente).

Méthode	Path	Auth	Description
GET	/push/config	X	{ enabled, publicKey } — clé VAPID publique pour s'abonner
POST	/push/subscribe	✓	Body : PushSubscription navigateur (endpoint, keys) — upsert par endpoint
DELETE	/push/subscribe	✓	Body : { endpoint } — désabonnement

Payload push : { title, body, data: { url, conversationId, productId }, tag } — l'utilisateur est redirigé vers /messages/:id ou /notifications au clic. Respecte User.notifications_push ; abonnements invalides supprimés (404/410).

Pour une app mobile native, les notifications passeraient par FCM/APNs (Firebase) — le serveur envoie alors vers un device token au lieu d'un endpoint Web Push. Le modèle PushSubscription stocke l'endpoint, p256dh, auth.